JARVIS

YouTube Agent — Codebase & Revision Guide

A practical reference for revisiting the module months later: architecture, execution flow, C++ concepts, Windows automation, persistence, error handling, testing, and known limitations.

Module purpose	Open the configured browser and search YouTube for a supplied command.
Primary platform	Windows, using `system()`, `Start`, WinApp CLI, and `findstr`.
Configuration	`Jarvis_Data/User_Main_Browser.txt`
Design style	Small functions with distinct responsibilities; the manager should call the agent rather than contain its internal implementation.

Prepared from the exact source code supplied in this conversation. Explanations distinguish what the code currently does from suggested future improvements.

Contents

- 1. Quick overview
- 2. What the module does and does not do
- 3. Files, headers, and C++ concepts
- 4. Function-by-function reference
- 5. Full execution flow
- 6. Flow diagram
- 7. Browser automation and command construction
- 8. Error handling and defensive checks
- 9. Configuration file lifecycle
- 10. How to build and integrate it
- 11. Test checklist
- 12. Known limitations and future improvements
- 13. Revision questions
- 14. Quick reference sheet

1. Quick overview

The YouTube agent receives a search phrase from the rest of JARVIS, determines which supported browser the user selected, launches that browser, waits five seconds, checks for a matching window, then uses WinApp CLI to focus the address bar and submit a YouTube search URL.

It is a semi-automatic YouTube feature: it opens the search-results page. It does not choose a video, click a result, or play a specific video by itself.

Input / dependency	Role
filtered_command	Search text already prepared by the caller/parser.
Jarvis_Data/User_Main_Browser.txt	Stores one browser key: brave, chrome, msedge, or firefox.
Windows Start command	Requests that Windows launch the selected browser.
WinApp CLI	Sends keyboard input and lists windows.
findstr	Checks whether a browser-name string appears in the window list.

Important: this source uses Windows-specific commands. It is not a portable cross-platform implementation.

2. What the module does and does not do

It does

- Ask for the primary browser the first time the configuration file is missing.
- Normalize the typed browser name to lowercase and map it to an internal key.
- Persist the choice in a text file for later runs.
- Launch the selected browser using a Windows Start command.
- Wait for five seconds, then check for a matching window using WinApp CLI and findstr.
- Send Ctrl+L, clear possible stray characters with two Backspaces, type a YouTube search URL, and press Enter.
- Offer a fallback prompt when the saved browser value is not recognized by the execution branches.

It does not

- Install a browser or install/configure WinApp CLI.
- Verify that YouTube has fully loaded or that the search results are visible.
- Select a video or guarantee that a video starts playing.
- Fully URL-encode every possible character in the search query.
- Check every system() return value; most automation commands are issued without checking their individual result.

3. Files, headers, and C++ concepts

Header	Why it is included in this source
iostream	Console prompts and messages through cout/cin.
cstdlib	system() to invoke Windows commands.
vector	Included but not used in the pasted code; can likely be removed.
string	Stores browser names, search queries, and command strings.
filesystem	fs::path, exists(), is_empty(), create_directory().
algorithm	replace() and transform().
fstream	ifstream-style/fstream reading and ofstream writing.
thread	this_thread::sleep_for().
chrono	chrono::seconds(5).

Concepts to remember

- Function declarations: the three prototypes tell the compiler that these functions exist before their definitions are encountered.
- Pass by value: strings and fs::path parameters are copied into the function. For these small inputs this is simple, though const references could reduce copies in a later cleanup.
- namespace alias: namespace fs = filesystem; lets the code write fs::path instead of filesystem::path.
- Filesystem paths: folder / filename combines path components without manually concatenating slash characters.
- Streams: fstream reads the saved browser; ofstream writes the selected browser.
- String search: find("brave") != string::npos means the substring was found.
- Algorithms: transform(..., ::tolower) lowercases the input; replace(...) changes spaces to plus signs.
- system() and c_str(): system() receives a C-style character string, so command.c_str() exposes the string's character data in the expected form.

4. Function-by-function reference

4.1 Youtube_Command_Executer(string filtered_command)

Purpose: main entry point for the YouTube feature. It reads configuration, launches the browser, builds a search URL, verifies the window, and sends keyboard actions.

Important local variables:

Variable	Meaning
user_primary_browser	Browser key loaded from the configuration file.
folder	Path object for Jarvis_Data.
p	Path object for Jarvis_Data/User_Main_Browser.txt.
base_url	Fixed YouTube search-results URL prefix.
query	Copy of filtered_command; spaces are replaced with +.
final_search_text	base_url + query, sent to the browser address bar.

Variable	Meaning
is_browser_opened	Exit status returned by the system() pipeline that invokes findstr.

Execution stages: configuration check → read browser key → launch → wait five seconds → construct URL → select browser-specific automation branch → check window → send keyboard input.

4.2 Browser_Detection_Module()

Purpose: ask the user to choose a supported browser if the configuration file does not exist, normalize the response, and save the mapped key.

Mapping performed by the code:

User text contains	Saved key
brave	brave
chrome	chrome
edge	msedge
firefox	firefox

The matching is substring-based and case-insensitive because the input is transformed to lowercase first. Typing the browser name is expected; the displayed numbers are currently menu labels only, not numeric choices.

If the response is empty, unsupported, or contains “other”, the function prints a message and returns without deliberately saving a new choice. The caller checks whether the file now exists and returns if it still does not.

4.3 Browser_Launch(string user_primary_browser)

Maps the saved browser key to a Windows command:

```
system("Start brave");
system("Start chrome");
system("Start firefox");
system("Start msedge");
```

The function prints a failure message if none of its browser-name checks matches. However, the function returns void, and the caller does not receive an explicit success/failure result. The caller relies on the later window check as a practical check.

4.4 Fallback_Module(fs::path User_Main_Browser_file_path)

Purpose: when the saved browser key reaches the final else in the execution function, ask for a replacement browser name and overwrite the same configuration file.

The function validates empty input, lowercases it, rejects “other”, maps supported names, and only reports successful replacement inside the file-open success branch. If the file cannot be opened, it prints a failure message.

Important nuance: this fallback is called for an unrecognized non-empty saved value. An empty file is handled earlier by fs::is_empty(p), which returns before the fallback is reached.

5. Full execution flow

- Start: caller passes filtered_command to Youtube_Command_Executer().
- Configuration exists? If not, call Browser_Detection_Module().
- Configuration created? If still missing, return without attempting automation.

- Configuration file empty? Print a message and return.
- Read configuration: getline() loads the browser key into user_primary_browser.
- Launch: Browser_Launch() asks Windows to open the configured browser.
- Wait: sleep for five seconds to allow a window to appear.
- Build URL: replace spaces in filtered_command with + and append it to the YouTube search URL prefix.
- Choose branch: inspect the browser key and run the matching Brave, Chrome, Edge, or Firefox branch.
- Window check: list windows and search for the expected browser title text using findstr.
- If no match: print a launch-delay message and return.
- If match: focus the address bar with Ctrl+L, send two Backspaces, type the URL, press Enter, then return.
- Unknown browser key: print a configuration warning, call Fallback_Module(), and return.

6. Flow diagram

Use this as a mental map when revisiting the code. Each decision corresponds to an if/else block in the source.

```
Caller
  |
  v
Youtube_Command_Executer(filtered_command)
  |
  v
Does config file exist? ---- no ----> Browser_Detection_Module()
  |                                      |
 yes                                     v
  |                                 file created?
  |                                   no | yes
  |                                      | (continue)
  |<-------------------------------------+
  v
Is file empty? ---- yes ----> print message and return
  |
 no
  v
Read browser key
  |
  v
Browser_Launch(key)
  |
  v
Wait 5 seconds; build YouTube search URL
  |
  v
Choose browser branch (Brave / Chrome / Edge / Firefox)
  |                                      |
  | recognized                            | unknown key
  v                                       v
list-windows | findstr               Fallback_Module()
  |
  v
Window match found?
  | no                          | yes
  v                             v
print delay message        Ctrl+L -> Backspace x2
return                     -> type URL -> Enter -> return
```

7. Browser automation and command construction

7.1 The URL

```
string base_url = "https://www.youtube.com/results?search_query=";
string query = filtered_command;
replace(query.begin(), query.end(), ' ', '+');
string final_search_text = base_url + query;
```

For example, the phrase `apna college c++` becomes `apna+college+c++` and is appended to the YouTube search URL. This is a simple space replacement, not complete URL encoding. Characters such as &, #, %, and some non-ASCII characters may require proper percent-encoding in a future version.

7.2 Window check: findstr

```
system("winapp ui list-windows | findstr /i \"Brave\" > nul");
```

The pipeline runs the window-list command, passes its text to findstr, searches case-insensitively for the word Brave, and hides output by redirecting it to nul. In a normal Windows command pipeline, a zero exit status indicates that findstr found a match; nonzero means it did not find a match.

This checks for a matching listed window after the fixed five-second wait. It does not prove that the browser is fully loaded, focused, or ready for keyboard input.

7.3 Keyboard steps

Command	Intended effect
Ctrl+L	Focus the address bar.
Backspace twice	Workaround for the observed issue where a stray “l” could be typed after the shortcut.
Send final_search_text	Enter the YouTube search URL.
Enter	Navigate to the URL.

The four browser branches repeat the same sequence but use different WinApp CLI application identifiers and window-name strings.

8. Error handling and defensive checks

Check / technique	What it protects against	What it does not guarantee
!fs::exists(p)	Avoids reading a configuration path that does not exist.	Does not guarantee setup succeeded.
Second exists(p) check	Returns if setup did not create the file (for example, permissions issue).	Does not identify the exact failure reason.
fs::is_empty(p)	Catches a zero-length configuration file before reading.	Does not catch a file containing only whitespace or an invalid bro
file.is_open()	Avoids reading/writing through a stream that failed to open.	The failed branch does not print an error if opening fails.
Input validation	Rejects empty, unsupported, or “other” browser names during setup.	Does not validate unexpected text that contains a s
Five-second wait	Gives the browser time to create a window.	A slow launch may need more than five seconds; a fast launch st
findstr exit status	Detects whether a matching browser label appears in the listed windows.	Does not guarantee the correct page loaded or successful keyboard actions.
Fallback_Module	Lets the user replace an unrecognized saved browser key.	Not reached for a zero-byte file, because the empty-file check re

Important failure paths

- Empty first-run input: Browser_Detection_Module() returns without creating a file; caller sees that the file is still missing and returns.

- Invalid first-run name: function prints a warning and returns; caller returns after the second existence check.
- Zero-byte config: main function prints configuration messages and returns before reading.
- Unknown non-empty config value: browser branches do not match, so the fallback asks for a replacement.
- Window not found after five seconds: a warning is printed and the function returns without typing the search URL.
- Config file cannot be opened for fallback writing: the latest code prints “Failed to Update Browser Configuration”.

Subtle issue: if the config file exists but cannot be opened for reading, user_primary_browser remains empty. The current code then attempts Browser_Launch(""), waits, and eventually enters the fallback. A more explicit read-open check would make this failure easier to diagnose.

9. Configuration file lifecycle

```
Jarvis_Data/
■■■ User_Main_Browser.txt
```

The file stores one plain-text browser key, for example `brave`. It is created on first run after a supported browser name is recognized. On later runs, the module reads the value and does not ask again.

To change the default browser manually, close JARVIS and edit the text file so it contains exactly one supported key: `brave`, `chrome`, `msedge`, or `firefox`. The fallback can also overwrite an invalid non-empty value.

The code uses a relative path. Therefore, Jarvis_Data is resolved relative to the program's current working directory, not necessarily the directory where the source file is stored. Launching the executable from a different working directory may make the configuration appear to be missing.

10. How to build and integrate it

This source defines the YouTube agent functions but does not contain main(). Compile it with the rest of the JARVIS source files that call it, and make sure the declaration is available to the caller through the corresponding header.

```
g++ -std=c++17 Manager.cpp temp_cleaner.cpp youtube_agent.cpp command_parser.cpp -o jarvis
```

This is an example command consistent with the project layout discussed for JARVIS; adjust source filenames if your local project differs. C++17 is needed for std::filesystem.

Integration contract

- The caller should pass the already-filtered YouTube search text into Youtube_Command_Executer().
- Keep browser setup, persistence, launching, window checks, and keyboard automation inside the YouTube module.
- Manager.cpp should remain the coordinator and call the module rather than duplicate its internals.
- The implementation assumes Windows commands and WinApp CLI are installed and available through PATH (or otherwise callable).

11. Test checklist

Test	Expected behavior
First run: type brave	Config file is created with brave; browser launch/search flow proceeds.
Second run	Saved key is reused without asking for the browser again.
First run: press Enter on prompt	Empty-input warning; no config file is created by detection.
First run: type opera	Invalid-choice warning; no supported key is saved.
Config file is zero bytes	Empty-file message; no launch/search attempt.
Config file contains unknown non-empty text	Fallback asks for a replacement browser name.
Browser launch takes longer than five seconds	Window check may fail and print the delay message.
Browser window is already open	A matching listed window may allow the automation to proceed.
Search text contains punctuation	Check whether the URL is interpreted correctly; only spaces are transformed.
WinApp CLI command fails	Observe whether the window check catches it; keyboard command return codes are curren

Test one scenario at a time and inspect the saved text file after setup/fallback. That separates configuration problems from browser automation problems.

12. Known limitations and future improvements

- Repeated branch code: four browser branches repeat nearly identical keyboard logic. A helper function could accept app selector and window-name text, but refactoring is optional after the current version is stable.
- Fixed sleep: five seconds is simple but not adaptive. A polling loop with a timeout could check for the window repeatedly without always waiting the full five seconds.
- Command return values: most system() calls for launching and keyboard automation are not checked. Future code could report which specific command failed.
- URL encoding: replace spaces with plus signs only; use a proper URL-encoding function if broader search input must be supported.
- Shell command construction: final_search_text is concatenated into a command passed to system(). Special characters or shell metacharacters can break the command and should be handled carefully.
- File-system exceptions: filesystem operations can throw exceptions in some failure cases. Error-code overloads or try/catch can make failure reporting more controlled.
- Configuration read validation: explicitly check whether the input stream opened and whether a supported key was actually read.
- Menu mismatch: the prompt shows numbered options but code expects browser names. The prompt says “Enter the Name,” and invalid input says to enter the exact name; numeric choices such as 1 are not implemented.
- Unused header: vector is included but not used in this source.

These are improvement ideas, not a claim that the existing architecture must be rewritten. Keep the current feature testable, then improve one reliability issue at a time.

13. Revision questions

- Why does Youtube_Command_Executer() check fs::exists(p) both before and after Browser_Detection_Module()?

- Why is the saved browser key normalized to lowercase?
- What does string::npos mean in a find() comparison?
- Why is fs::path used instead of manually writing a path with backslashes?
- What is the role of the five-second sleep, and what are its weaknesses?
- What does findstr return when it finds a match? Why is output redirected to nul?
- Why are two Backspace commands sent after Ctrl+L?
- Why does an empty file return before the fallback branch?
- What is the difference between the browser being open and the browser being ready?
- Why can special characters in filtered_command cause URL or shell-command problems?
- Which system() calls have their return values checked, and which do not?
- Why is the configuration path relative to the current working directory?

Suggested self-test

Try explaining the full flow aloud without looking at the code. Then trace these three cases on paper: (1) first run with brave, (2) an empty configuration file, and (3) an unknown non-empty configuration value. If you can explain each return point and why it happens, you understand the main control flow.

14. Quick reference sheet

Item	Remember
Entry point	Youtube_Command_Executer(filtered_command)
Config path	Jarvis_Data/User_Main_Browser.txt
Saved values	brave / chrome / msedge / firefox
Launch commands	Start brave / chrome / firefox / msedge
Wait	this_thread::sleep_for(chrono::seconds(5))
Window check	winapp ui list-windows | findstr /i "BrowserName" > nul
Address bar	Ctrl+L, then two Backspaces
Search URL prefix	https://www.youtube.com/results?search_query=
Query transform	spaces become +
Fallback trigger	non-empty browser key that matches no supported branch
Build standard	C++17 or later for std::filesystem

End of guide. Source behavior is described as provided; future improvements are marked separately.